

Quando um sistema com LLM vira agente

Na aula anterior, o sistema ganhou ferramentas. O modelo podia solicitar uma consulta, acionar uma função, pedir uma API ou devolver uma intenção estruturada para que um orquestrador executasse uma ação real. Essa mudança já é importante: a IA deixa de produzir apenas texto e passa a participar de um fluxo operacional. Esse avanço, isoladamente, ainda não caracteriza um agente.

Considere um assistente que recebe uma solicitação sobre a cotação atual do dólar. O modelo percebe que não tem esse dado em sua memória interna, solicita a ferramenta de cotação, o orquestrador executa a chamada e devolve o resultado. O modelo, então, monta a resposta final. Esse sistema é útil, mas continua basicamente reativo. Ele responde a uma solicitação, usa uma ferramenta quando necessário e encerra o processo.

Um agente acrescenta outra camada. Ele não apenas reage a uma solicitação: ele trabalha em direção a um objetivo. Para isso, precisa acompanhar o que já tentou, observar os resultados, decidir o próximo passo e saber quando parar. A diferença parece pequena no vocabulário, mas é grande na arquitetura.

Antes de chamar de agente

Alguns termos aparecem juntos quando falamos de IA aplicada, mas eles não dizem a mesma coisa. Um **workflow** é uma sequência definida previamente: etapa A, depois B, depois C. Um **sistema com contratos** adiciona controle a essa sequência, validando entradas e saídas para que cada parte seja testável. **Tool calling** permite que o modelo solicite ações externas, como consultar uma API ou executar uma função. Um **agente**, por sua vez, usa essas capacidades dentro de um ciclo orientado a objetivo. Quando esse trabalho é dividido entre papéis especializados, temos uma arquitetura **multiagente**. Essa distinção evita uma confusão comum. Se um sistema chama uma ferramenta, ele pode fazer parte de uma arquitetura agente, mas isso não o transforma automaticamente em agente. A caracterização como agente depende de o sistema acompanhar seu progresso e decidir a próxima ação com base no que observou.

O ciclo essencial

Um agente útil precisa de um ciclo explícito:

planejar -> gerar -> avaliar -> corrigir -> repetir

Esse ciclo não é uma decoração conceitual. Ele muda a forma como o software é desenhado. Em vez de produzir uma resposta única, o sistema passa a trabalhar em rodadas. Em cada rodada, ele precisa entender qual é o objetivo atual, escolher uma ação compatível com esse objetivo, observar o resultado da ação e comparar esse resultado com algum padrão de qualidade.

Em um agente que deve gerar um PRD para um aplicativo de acompanhamento de estudos, a primeira tentativa pode resultar em um documento genérico, com menções a tarefas, lembretes e painel de progresso. Ao avaliar essa saída, o sistema identifica lacunas como ausência de personas, métricas de sucesso e critérios de aceite. A rodada seguinte não começa do zero: ela parte dos problemas identificados. O agente reescreve o PRD tentando corrigir essas lacunas. Se a nova versão ainda for insuficiente, o ciclo continua.

Sem esse ciclo, temos apenas uma sequência de prompts. Com o ciclo, começamos a ter um sistema que aprende operacionalmente durante a execução, ainda que esse aprendizado seja local, limitado ao estado daquele processo.

Estado: a memória operacional do agente

O estado é uma das diferenças centrais entre uma chamada isolada e um agente. Em uma chamada comum, o sistema recebe uma entrada e devolve uma saída. Em um agente, a saída de uma rodada influencia a próxima. Para isso acontecer, o sistema precisa guardar informações sobre o caminho percorrido.

No agente gerador de PRD, o estado deve registrar o objetivo do produto, a iteração atual, as versões produzidas, a avaliação de cada versão, os problemas encontrados e a decisão de parada. Essa memória operacional permite que o agente compare versões. Sem ela, o sistema apenas gera de novo e espera que o novo texto seja melhor. Com ela, o sistema sabe o que estava errado, o que foi alterado e por que decidiu continuar ou encerrar.

Um estado mínimo poderia ser representado assim:

```
{
  "objetivo": "Gerar um PRD para um aplicativo de acompanhamento de estudos",
  "iteracao_atual": 2,
  "limite_iteracoes": 4,
  "qualidade_minima": 8,
  "versoes_prd": [
    {
      "versao": 1,
      "conteudo": "...",
      "nota": 6.5,
      "problemas": ["faltam métricas", "personas genéricas"]
    },
    {
      "versao": 2,
      "conteudo": "...",
      "nota": 8.2,
      "problemas": ["escopo técnico ainda superficial"]
    }
  ],
  "status": "aprovado"
}
```

Esse registro não serve apenas para o computador. Ele também serve para o aluno, para o professor e para a equipe que vai confiar no sistema. Um agente sem histórico é difícil de auditar. Um agente com estado claro permite reconstruir o raciocínio operacional: qual versão foi gerada, o que havia de errado nela, que correção foi tentada e qual regra justificou o encerramento.

Critério de parada

Todo agente precisa saber quando parar. Sem critério de parada, o sistema pode entrar em loop, gastar recursos sem necessidade ou piorar uma solução que já estava adequada. A autonomia, nesse caso, deixa de ser uma vantagem e passa a ser um risco.

No exemplo do PRD, um critério simples seria:

Parar quando a nota for maior ou igual a 8 e não houver problema crítico, ou quando forem atingidas 4 iterações.

Esse tipo de regra combina qualidade e segurança. A nota mínima força o agente a buscar um resultado aceitável. O limite de iterações impede que ele continue indefinidamente. Em sistemas reais, outros limites podem entrar no desenho, como custo máximo, tempo máximo, aprovação humana ou ausência de divergência entre revisores.

O limite de iterações não é um detalhe burocrático. Ele é uma proteção contra a falsa sensação de melhoria. Modelos podem reescrever um texto de forma mais convincente sem corrigir o problema principal. Por isso, um agente responsável não deve apenas produzir versões novas; ele deve registrar por que acredita que a versão final é suficiente.

Agente não é ausência de orquestração

Outro erro comum é associar agente a "deixar o modelo fazer tudo sozinho". Na prática profissional, costuma acontecer o oposto. Quanto maior for o grau de autonomia atribuído ao sistema, maior precisa ser o cuidado com a orquestração.

O modelo pode propor um plano, gerar uma versão, avaliar uma resposta, sugerir correções e escolher uma próxima ação dentro dos limites definidos. Mas o código continua tendo responsabilidades que não podem ser entregues ingenuamente ao modelo: manter o estado, validar formatos, executar ferramentas reais, aplicar limites, registrar decisões, bloquear ações perigosas e pedir aprovação humana quando necessário.

Portanto, a autonomia do agente não está fora do controle. Ela acontece dentro de uma arquitetura de controle. O agente decide, mas decide em um espaço delimitado pelo sistema.

Multiagentes

Em uma arquitetura multiagente, o trabalho é dividido por papéis. Essa divisão só faz sentido quando cada papel tem uma responsabilidade real. Para gerar um PRD, por exemplo, podemos separar três funções: um gerador, que escreve ou reescreve o documento; um crítico, que identifica lacunas de clareza, completude e coerência; e um revisor técnico, que verifica viabilidade, riscos e critérios de aceite.

O valor dessa divisão está na tensão produtiva entre os papéis. Quem gera tende a expandir e organizar ideias. Quem critica procura falhas. Quem revisa tecnicamente protege o documento contra promessas inviáveis. Se todos os agentes apenas repetirem a mesma tarefa com nomes diferentes, a arquitetura fica mais complexa sem ficar mais inteligente.

O mesmo padrão pode ser levado para outras situações. Em um agente de melhoria de código, um papel pode analisar o erro, outro pode aplicar a correção e outro pode verificar testes e regressões. Em um agente que acompanha a evolução de um TCC, um orientador pode olhar a coerência geral do trabalho, um crítico pode testar a força do argumento e um revisor metodológico pode examinar problema, objetivos, método e instrumentos de pesquisa.

Mesmo em sistemas multiagentes, o orquestrador continua essencial. Ele define a ordem das interações, atualiza o estado compartilhado, controla limites e decide se o processo deve continuar. Os agentes contribuem com perspectivas diferentes; o orquestrador transforma essas contribuições em processo.

A medida da autonomia

A autonomia de um agente deve ser avaliada por critérios de arquitetura, não por uma noção genérica de inteligência. Para fins de projeto, a formulação técnica é:

Um sistema agente persegue um objetivo, observa o resultado das próprias ações, usa estado para melhorar e para por um critério definido.

Quando essas propriedades estão presentes, estamos diante de uma arquitetura agente. Quando elas não estão, provavelmente temos um workflow, uma cadeia de prompts ou um sistema com ferramentas. Todos podem ser úteis. A nomeação correta ajuda a escolher controles, limites e formas de validação adequadas.

Para o aluno, essa distinção orienta o projeto técnico. Construir agentes não consiste em escrever prompts mais longos nem em empilhar ferramentas em volta de um modelo. Consiste em desenhar um processo com objetivo, memória operacional, avaliação, correção e encerramento responsável. Nessa arquitetura, a IA deixa de ser apenas uma resposta e passa a participar de um sistema capaz de trabalhar sobre o próprio resultado.